Code review — colections-csharp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 3 — 2026-10-20 — sha `2db692a`

Scop: fix-urile din runda 2 + Pas 3 (`ColectieAbstracta`), Pas 4 (`StivaTablou`, `CoadaCirculara`) și începutul de la Pas 5 (`DictionarHash`), commit-ul `2db692a` „code review + stiva + coada + start dictionar".

Metodă: build pe .NET 8. Fiindcă nu compilează (B1), am scos linia 125 pe o copie, am chemat `Pas1` – `Pas4` una după alta și am adăugat 19 teste mici: liste goale și pline, `Remove` / `RemoveAt` la capete, `Clear`, `Count` din afară, coada care se rotește și apoi se mărește, 100.000 de `Add`. Codul tău nu a fost atins.

Poarta de compilare: ❌ **1 eroare** (`Program.cs:125`), ⚠️ **25 de warning-uri** (runda 2: 14).

Pe scurt:

- ✅ **Runda 2, pe fond:** `ToString` și `ToArray` din `ListaInlantuita` sunt corecte, `Tail` din `Remove` e reparat, `GetEnumerator`-ul ne-generic merge, iar `EqualityComparer` e acum în **toate cele 4 clase**. E prima regulă aplicată peste tot, nu doar unde ți-am arătat-o. 👏
- ✅ **Coada circulară e gândită bine:** `Pop` fără nicio buclă, `% items.Length` peste tot, iar după rotire + mărire ordinea rămâne corectă (`[4, 5, 6, 7, 8, 9]`). Stiva și coada aruncă `InvalidOperationException` pe goală.
- 🔴 Dar `Main` nu mai cheamă niciun pas, așa că n-ai văzut că Pas 1 și Pas 2 s-au stricat la mutarea în `ColectieAbstracta` (B2, B3, B4), iar Pas 3 iese corect doar din noroc.
- 🔴 Dublarea capacității a dispărut din toate trei clasele cu tablou (B5): asta e exact cazul de 5 miliarde de copieri din fișă.

Ieșirea, comparată cu fișa

Pas	Rând din fișă	La tine	De ce
1	`b;D;f;`	*(rând gol)*	B2
2	`[] Count=0`	`[,,,] Count=0`	B3
2	`True False` și restul	`IndexOutOfRangeException` la `ListaTablou.cs:79`	B4
3	toate 6 rândurile	identice (fără spațiul după virgulă)	✅ din noroc, vezi B3
4	`[e, d, c, b, a] Varf=e`	`[a,b,c,d,e] Varf=e`	B6
4	`e d [c, b, a]`	`e d [a,b,c]`	B6
4	`[40, 50, 60, 70, 80] Capacitate=8`	`... Capacitate=4`	B5

Pe toate rândurile lipsește spațiul după virgulă (C2).

🔴 Critice

B1 — `Program.cs:125`: proiectul nu compilează

Dovada: `error CS1002: ; expected`. În `Main` a rămas un `IDictionary` început, iar toți pașii sunt comentați.

Mecanism: compilatorul nu poate construi un singur `.exe` din bucăți. O singură eroare, oriunde în proiect, înseamnă că nu rulează nici `ListaInlantuita`, care e corectă. De-aia un commit trebuie să compileze întotdeauna, chiar dacă e „start dictionar”.

💡 Ce lucrezi încă stă într-o metodă `Pas5()` pe care `Main` n-o cheamă deocamdată. `Main` cheamă `Pas1()`–`Pas4()` una după alta, cum cere fișa.

B2 — `ListaInlantuita.cs:11`: două `Count`-uri, iar `foreach` îl citește pe cel greșit

Dovada: pe `[a, b]`, `foreach` nu dă nimic. `li.Count` = 2, dar `((ColectieAbstracta<string>)li).Count` = 0, iar `li.Count()` din LINQ = 0. În Pas 1, rândul `b;D;f;` iese gol.

Mecanism: `ColectieAbstracta` are deja un `Count`. Scriind încă unul în `ListaInlantuita`, nu l-ai înlocuit, ci l-ai **ascuns** (warning-ul `CS0108` spune exact asta). Acum sunt două proprietăți cu același nume. `Add` din listă îl crește pe al ei, iar `GetEnumerator` din clasa de bază îl citește pe al lui, care a rămas 0. Deci bucla `for (i = 0; i < Count; ...)` nu face niciun pas. În `Program`, `lista.Count` trece prin interfață și îl găsește pe cel nou, așa că afișarea pare corectă.

Mai e o problemă cu același `Count`: în bază are `set` public (`ColectieAbstracta.cs:9`). Dovada: `lista.Count = 10` din `Program`, apoi `foreach` → `IndexOutOfRangeException` la `:33`.

💡 Fișa, Pas 3: `Count` e `abstract` în bază. Ce-ar face compilatorul cu un `Count` scris de două ori dacă în bază ar fi `abstract`?

B3 — `ListaTablou.cs:48–51`: `ToArray` întoarce tabloul din interior

Dovada: `[1, 2]` → `ToArray()` = `1|2|0|0`, `Contains(0)` = `True`. Iar `t = lista.ToArray(); t[0] = 99` **schimbă lista**: `[99,2,0,0]`. În Pas 2, lista goală se afișează `[,,,]`.

Mecanism: un tablou e o referință. `return items` nu face o copie, ci dă afară adresa tabloului tău, cu tot cu locurile goale de la capăt. E aceeași regulă ca la B2 din runda 2 („un tablou cu `Count` elemente, nici unul în plus”), reparată în `ListaInlantuita` și repetată aici.

De ce trece totuși Pas 3? Fiindcă B5 mărește tabloul cu exact 1: după al 5-lea `Add` are fix 5 locuri, deci tabloul intern e plin și „pare” o copie corectă. Două bug-uri care se anulează.

💡 `StivaTablou.ToArray` (`:42–52`) e scris corect. Ce face în plus?

B4 — `ListaTablou.cs:31`, `:79`: golirea locului crapă când tabloul e plin

Dovada: `[a, b, c, d]` → `Remove("a")` → `IndexOutOfRangeException` la `:31`. În Pas 2 crapă la `:79` (`RemoveAt`). Iar pe `[a, b, c]`, `RemoveAt(3)` (adică `Count`) **șterge ultimul element**: `Count=2`, `c` dispare.

Mecanism: după mutare, ultimul element ocupat e la `Count - 1`. `items[Count]` e primul loc **liber**, iar când tabloul e plin, locul ăsta nu există. Cu B5 tabloul e aproape mereu plin, deci crapă aproape mereu. A doua problemă e garda de la `:73`: `index > Count` lasă să treacă `index == Count`. Bucla nu face nimic, dar `Count--` se execută oricum.

💡 În `StivaTablou.cs:26` și `CoadaCirculara.cs:27` ai scris aceeași linie corect. Compară-le indicele. 💡 Garda din `ListaInlantuita.RemoveAt` (`:135`) e corectă. Ce semn are acolo?

B5 — `ListaTablou.cs:109`, `StivaTablou.cs:81`, `CoadaCirculara.cs:86`: tabloul crește cu 1, nu se dublează

Dovada: 100.000 de `Add` = **4.999.949.994 de copieri, 14 secunde** (fișa: 131.068 cu dublare). `Capacity` minte în toate trei clasele:

	`Capacity`	tabloul real
`ListaTablou`, după 9 `Add`	16	9
`ListaTablou`, `Clear` + 5 `Add`	0	5

	Capacity	tabloul real
`StivaTablou`, după 9 `Push`	4	9
`CoadaCirculara`, Pas 4	4	5

Pas 2 îți afișează `4 → 8 → 16`, deci pare corect, dar tabloul nu e nici 8, nici 16.

Mecanism: `new T[Count + 1]` face loc pentru **un** element. Următorul `Add` găsește tabloul iar plin și copiază totul din nou: 1 + 2 + 3 + … + n ≈ n²/2 copieri. Dublarea copiază tot, dar rar: după o mărire urmează atâtea `Add`-uri gratuite câte elemente ai copiat. `Capacity` a rămas număr scris de mână lângă tablou, adică B5 din runda 2, iar acum nici nu mai descrie același tablou.

💡 Fișa, Pas 2, punctul 2: `Capacitate` = lungimea tabloului, doar `get`. Dacă e scrisă `=> items.Length`, mai poate minți? 💡 Cu cât trebuie să fie mai mare tabloul nou ca să iasă 4 → 8 → 16?

B6 — `StivaTablou`: `foreach` merge de la bază spre vârf

Dovada: după `Push` a, b, c, d, e, stiva se afișează `[a,b,c,d,e]` (fișa: `[e, d, c, b, a]`). Iar pe `a b a b`, `Remove('a')` scoate `a`-ul de la bază, nu pe cel de lângă vârf.

Mecanism: `GetEnumerator` din bază parcurge ce-i dă `ToArray`, iar `StivaTablou.ToArray` copiază în ordinea din tablou, de la 0 în sus. Pentru listă ordinea asta e corectă. Pentru stivă nu e: ordinea naturală a stivei e cea în care ies elementele, de la vârf. Template Method lasă exact partea asta fiecărei clase.

💡 Fișa, Pas 3: `GetEnumerator` generic e **`abstract`**, adică fiecare colecție spune singură în ce ordine se parcurge. Unde ar trebui să înceapă bucla pentru stivă?

🟡 Importante

- **M1** — numele din contract, rămas de decis la ședință: `Add` / `Remove` în loc de `Adauga` / `Sterge`. Acum se vede și un cost: `ICoada` are aceleași `Push` / `Pop` / `Peek` ca `IStiva`, deci o coadă se citește ca o stivă (fișa: `Intra` / `Iese` / `Primul`). Iar `Main` nu mai cheamă **niciun** pas: așa au trecut B2, B3 și B4 neobservate.
- **M2** — Template Method e întors pe dos. Fișa: în bază, `Contains`, `ToArray`, `ToString` scrise cu `foreach (T x in this)`, iar `GetEnumerator` și `Count` lăsate `abstract`. La tine, baza construiește `foreach` din `ToArray`: fiecare parcurgere și fiecare `Contains` copiază întâi toată colecția. Iar listele au încă `GetEnumerator`-ul ne-generic (`ListaInlantuita.cs:236`, `ListaTablou.cs:102`), deși e deja în bază (`ColectieAbstracta.cs:37`). „Gata când” de la Pas 3 cere să dispară.
- **M3** — `Node<T>` e tot `public` în namespace, cu câmpuri publice, iar warning-urile au crescut de la 14 la 25 (`ListaInlantuita` 10, `DictionarHash` 6, `Node` 3, `ListaTablou`, `StivaTablou`, `CoadaCirculara` câte 2). Unul dintre ele, `CS0108`, era chiar B2.
- **M4** — indexul greșit, tot tăcut sau crud: `ListaTablou[5]` pe `Count=5` → `IndexOutOfRangeException` la `:98`, `ListaInlantuita.Insert(5, "x")` pe lista goală → `[x]`. Pas 6 le aduce la `ArgumentOutOfRangeException`.
- **M5** — `DictionarHash` abia început, deci nu-l revizuiesc acum, cu o singură observație de structură: `key`, `value`, `next` (`:8–10`) sunt câmpurile **unui nod**, puse pe dicționar. Iar `Node<K>[]` ține doar chei, deci valoarea n-are unde sta. 💡 Fișa, Pas 5: clasa internă `Nod` cu `Cheie`, `Valoare`, `Urmator`.

🟢 Cleanups

- **C1** — `Tail = new Node<T>()` în constructor (`ListaInlantuita.cs:16`) și `Node()` doar pentru el: rămase din runda 1.
- **C2** — `String.Join(", ", ...)` (`ColectieAbstracta.cs:39`): fișa are `", "`, de-aia nu se potrivește niciun rând cu virgulă.
- **C3** — `//ICollection`, `//IList`, `//IColectie`, `//ILista`, `ExYield` în `Program`: notițele merg în `RASPUNSURI.txt`, care încă lipsește. `CoadaCirculara` are câmpuri fără `private` (`:8–9`), spre deosebire de celelalte clase.

Q&A

1. `ListaInlantuita` are acum două `Count`-uri. Care din ele crește la `Add` și pe care îl citește `foreach`? De ce `lista.Count` din `Program` îți arăta totuși numărul corect?
2. Pas 3 iese corect pe `ListaTablou`, deși `ToArray` întoarce tabloul din interior. Ce bug îl ascunde? Ce-ar afișa Pas 3 dacă adaugi doar 3 pătrate în loc de 5?
3. În `StivaTablou` și `CoadaCirculara` golești locul de la `Count - 1`, în `ListaTablou` pe cel de la `Count`. Ce diferență e între cele două poziții?

Răspunsurile (și cele 6 din rundele 1 și 2) în `RASPUNSURI.txt`, în repo.

Următorul pas: B1 (compilează, `Main` cheamă `Pas1()`–`Pas4()`), apoi B2–B6 până când toate cele 4 ieșiri sunt identice cu fișa, rând cu rând, și abia apoi `DictionarHash`. Un test rapid pentru B5: 100.000 de `Add` trebuie să dureze sub o secundă.

Runda 2 — 2026-10-17 — sha `a8e794b`

Fix-urile din runda 1 + Pas 2, `ListaTablou<T>`.

- ✅ **B1** `ToString` dubla primul element: închis (mutat în `ColectieAbstracta`)
- ✅ **B2** `AsVector` cu `Count + 1`: închis în `ListaInlantuita` · 🔴 repetat în `ListaTablou.ToArray` (→ runda 3, B3)
- ✅ **B3** `Tail = Head` în `Remove`: închis
- ✅ **B4** `GetEnumerator` ne-generic în `ListaTablou`: închis
- 🔴 **B5** `Capacity` câmp separat de tablou: neînchis, mai grav (→ runda 3, B5)
- ✅ **B6** `Equals` pe `null`: închis, în toate cele 4 clase
- ⏸ **M1** numele din contract: de decis la ședință (→ runda 3, M1)
- 🔴 **M2** nod public + warning-uri: 14 → 25 (→ runda 3, M3)
- 🔴 **M3** index greșit tăcut: neatins (→ runda 3, M4)
- ✅ **M4** dublare scrisă de 2 ori, bucle de căutare: închis (`ResizeIfFull`)
- ⚠️ **C1** `Node()` + `Tail` gol în constructor: rămas · ⚠️ **C2** golirea locului: scrisă, dar pe indicele greșit (→ runda 3, B4) · 🔴 **C3** `RASPUNSURI.txt` lipsă

Runda 1 — 2026-10-16 — sha `ac84a76`

Pasul 1: `IColectie<T>`, `ILista<T>`, `ListaInlantuita<T>`.

- ✅ **B1** `foreach` / `GetEnumerator`: închis · ⚠️ `ToString` scris, dar dublează primul element (→ runda 2, B1)
- ✅ **B2** `RemoveAt(0)` scădea `Count` de două ori: închis · ⚠️ ascunderea lui prin `Count + 1` în `AsVector` a rămas (→ runda 2, B2)
- ✅ **B3** `Tail` după constructor, `Clear`, `Insert(0)`: închis · 🔴 o linie nouă greșită în `Remove` (→ runda 2, B3)
- ✅ **B4** listă goală și listă cu un element: închis
- ✅ **M1** `Head` folosit drept cursor: închis
- ⏸ **M2** numele din contract: de decis la ședință (→ runda 2, M1)
- 🔴 **M3** nod public + warning-uri: neatins, 13 → 14 (→ runda 2, M2)
- 🔴 **M4** index greșit tăcut: neatins (→ runda 2, M3)
- ✅ **C1** `using`-uri nefolosite · ⚠️ **C2** `Node()` rămas · 🔴 **C3** notițe în cod, `RASPUNSURI.txt` lipsă